Case Study 2

Evidence Collection

"What Companies Say vs. What They Do"

Labs 3-4 — Weeks 3-4

Course: Big Data and Intelligent Analytics
Instructor: Sri Krishnamurthy — QuantUniversity
Term: Spring 2026

Assigned: January 23, 2026
Due: February 6, 2026 (3:59 PM)

SEC filings tell you what companies *say*. External signals tell you what they *do*.

This case study builds the evidence collection pipeline—ingesting SEC documents and external signals that will feed the AI-readiness scoring engine.

Weight	8% of final grade
Format	Individual work
Submission	GitHub repository + README
Technologies	sec-edgar-downloader, pdfplumber, httpx, BeautifulSoup
Lab Foundation	Lab 3 (Week 3) + Lab 4 (Week 4)
Builds On	Case Study 1: Platform Foundation

Contents

1 Business Context **3**
1.1 Why This Matters . . . 3
1.2 The Target Companies . . . 3

2 Technical Architecture **5**
2.1 System Overview . . . 5
2.2 New Project Structure . . . 5

3 SEC EDGAR Pipeline (Lab 3) **6**
3.1 Filing Types and Their Value . . . 6
3.2 Using sec-edgar-downloader . . . 6
3.3 Document Parsing . . . 7
3.4 Semantic Chunking . . . 10
3.5 Document Registry . . . 12

4 External Signals Pipeline (Lab 4) **13**
4.1 Signal Data Models . . . 13
4.2 Job Posting Signal Collector . . . 14
4.3 Technology Stack Signal Collector . . . 16
4.4 Patent Signal Collector . . . 17

5 Database Schema Extensions **20**

6 API Endpoints **22**
6.1 Collection Endpoint Example . . . 22

7 Evidence Collection Script **24**

8 Evaluation Criteria **28**
8.1 Requirements Checklist . . . 28
8.2 Grading Rubric . . . 29

9 Submission Requirements **30**
9.1 What to Submit . . . 30
9.2 Evidence Report Template . . . 30
9.3 Submission Process . . . 31

10 Looking Ahead **32**

1 Business Context

Connection to CS1

Building on CS1: In Case Study 1, you built the platform foundation—API endpoints, data models, and persistence layer. Now you'll populate that platform with *evidence*: the raw data that will eventually be scored and analyzed.

Business Scenario

The partners have a question: "How do we know if a company is actually investing in AI, or just talking about it?"
Your platform needs two types of evidence:

1. **What companies SAY** (SEC Filings)
   - 10-K annual reports: Strategy, risk factors, MD&A
   - 10-Q quarterly reports: Recent developments
   - 8-K material events: AI announcements, executive changes
   - DEF-14A proxy statements: Executive compensation tied to tech
2. **What companies DO** (External Signals)
   - Job postings: Are they hiring AI/ML engineers?
   - Technology stack: What tools are they actually using?
   - Patents: Are they innovating in AI?
   - News/press releases: What are they announcing?

Your task: Build pipelines to collect both types of evidence for 10 target companies.

1.1 Why This Matters

Key Insight

The Say-Do Gap
Companies often overstate AI capabilities in public filings:

- 73% of companies mention "AI" in 10-K filings (up from 12% in 2018)
- But only 23% have deployed AI in production
- The gap between rhetoric and reality is measurable

External signals close this gap. A company claiming AI leadership but posting zero ML engineer jobs is telling you something important.

1.2 The Target Companies

You will collect evidence for these 10 companies across 5 sectors:

Table 1: Target Companies for Evidence Collection

Ticker	Company	Sector	Industry
CAT	Caterpillar Inc.	Industrials	Manufacturing
DE	Deere & Company	Industrials	Manufacturing
UNH	UnitedHealth Group	Healthcare	Healthcare Services
HCA	HCA Healthcare	Healthcare	Healthcare Services
ADP	Automatic Data Processing	Services	Business Services
PAYX	Paychex Inc.	Services	Business Services
WMT	Walmart Inc.	Consumer	Retail
TGT	Target Corporation	Consumer	Retail
JPM	JPMorgan Chase	Financial	Financial Services
GS	Goldman Sachs	Financial	Financial Services

Hint

These companies were selected because they span different AI maturity levels—from AI leaders (JPM, WMT) to traditional operators (CAT, HCA). This diversity will be valuable when you build the scoring engine in CS3.

2 Technical Architecture

2.1 System Overview

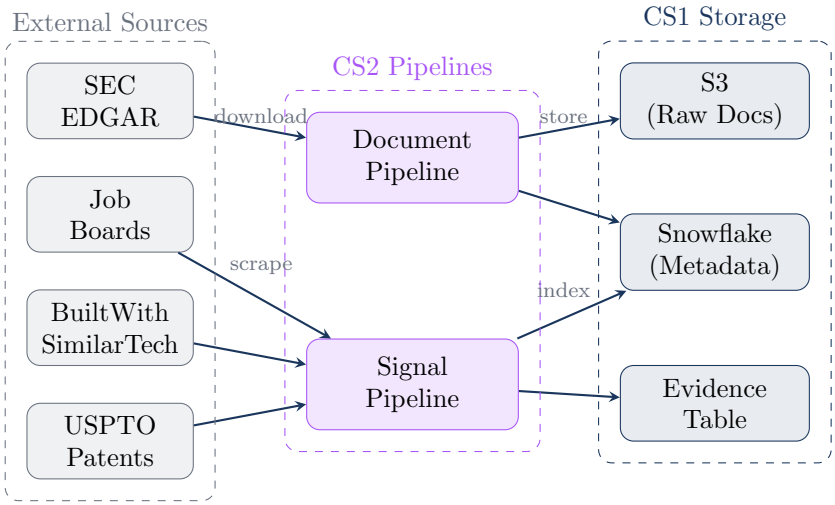

Figure 1: Evidence Collection Architecture

2.2 New Project Structure

Add these directories to your CS1 project:

```
pe-org-air-platform/
|-- app/
|   |-- pipelines/                    # NEW: Evidence pipelines
|   |   |-- __init__.py
|   |   |-- sec_edgar.py              # SEC filing downloader
|   |   |-- document_parser.py        # PDF/HTML extraction
|   |   |-- job_signals.py            # Job posting collector
|   |   |-- tech_signals.py           # Technology stack analyzer
|   |   |-- patent_signals.py         # Patent search
|   |-- models/
|   |   |-- document.py               # NEW: Document models
|   |   |-- evidence.py               # NEW: Evidence models
|   |   |-- signal.py                 # NEW: Signal models
|   |-- routers/
|   |   |-- documents.py              # NEW: Document endpoints
|   |   |-- signals.py                # NEW: Signal endpoints
|-- data/
|   |-- raw/                          # Downloaded documents
|   |-- processed/                    # Extracted text/chunks
|   |-- signals/                      # Signal cache
|-- tests/
|   |-- test_sec_edgar.py
|   |-- test_signals.py
|-- scripts/
|   |-- collect_evidence.py           # Main collection script
|   |-- backfill_companies.py         # Backfill for 10 companies
```

3 SEC EDGAR Pipeline (Lab 3)

3.1 Filing Types and Their Value

Table 2: SEC Filing Types for AI Evidence

Form	Frequency	AI-Relevant Sections
10-K	Annual	Item 1 (Business), Item 1A (Risk Factors), Item 7 (MD&A)
10-Q	Quarterly	Item 2 (MD&A), Item 1A (Risk Factors updates)
8-K	As needed	Item 8.01 (Other Events)—AI announcements
DEF-14A	Annual	Executive comp tied to technology metrics

3.2 Using sec-edgar-downloader

```python
from sec_edgar_downloader import Downloader
from pathlib import Path
import logging

logger = logging.getLogger(__name__)

class SECEdgarPipeline:
    """Pipeline for downloading SEC filings."""

    def __init__(
        self,
        company_name: str,
        email: str,
        download_dir: Path = Path("data/raw/sec")
    ):
        self.dl = Downloader(company_name, email, download_dir)
        self.download_dir = download_dir

    def download_filings(
        self,
        ticker: str,
        filing_types: list[str] = ["10-K", "10-Q", "8-K"],
        limit: int = 10,
        after: str = "2020-01-01"
    ) -> list[Path]:
        """
        Download filings for a company.

        Args:
            ticker: Company ticker symbol
            filing_types: List of filing types to download
            limit: Maximum filings per type
            after: Only filings after this date (YYYY-MM-DD)

```

```
        Returns:
            List of paths to downloaded filings
        """
        downloaded = []

        for filing_type in filing_types:
            try:
                self.dl.get(
                    filing_type,
                    ticker,
                    limit=limit,
                    after=after
                )

                # Find downloaded files
                filing_dir = self.download_dir / "sec-edgar-filings" / ticker / filing_type
                if filing_dir.exists():
                    for filing_path in filing_dir.glob("**/full-submission.txt"):
                        downloaded.append(filing_path)
                        logger.info(f"Downloaded: {filing_path}")

            except Exception as e:
                logger.error(f"Error downloading {filing_type} for {ticker}: {e}")

        return downloaded
```

Warning

SEC Rate Limits: The SEC limits requests to 10 per second. The `sec-edgar-downloader` library handles this, but be aware if you're making additional requests. Always include a valid email in your User-Agent.

3.3 Document Parsing

SEC filings come in multiple formats. You need to handle both:

```
import pdfplumber
from bs4 import BeautifulSoup
from pathlib import Path
from dataclasses import dataclass
from datetime import datetime, timezone
import hashlib
import re

@dataclass
class ParsedDocument:
    """Represents a parsed SEC document."""
    company_ticker: str
    filing_type: str
    filing_date: datetime
```

```python
    content: str
    sections: dict[str, str]
    source_path: str
    content_hash: str
    word_count: int

class DocumentParser:
    """Parse SEC filings from various formats."""

    # Regex patterns for section extraction
    SECTION_PATTERNS = {
        "item_1": r"(?:ITEM\s*1[.\s]*BUSINESS)",
        "item_1a": r"(?:ITEM\s*1A[.\s]*RISK\s*FACTORS)",
        "item_7": r"(?:ITEM\s*7[.\s]*MANAGEMENT)",
        "item_7a": r"(?:ITEM\s*7A[.\s]*QUANTITATIVE)",
    }

    def parse_filing(self, file_path: Path, ticker: str) -> ParsedDocument:
        """Parse a filing and extract structured content."""

        suffix = file_path.suffix.lower()

        if suffix == ".pdf":
            content = self._parse_pdf(file_path)
        elif suffix in [".htm", ".html", ".txt"]:
            content = self._parse_html(file_path)
        else:
            raise ValueError(f"Unsupported file type: {suffix}")

        # Extract sections
        sections = self._extract_sections(content)

        # Generate content hash for deduplication
        content_hash = hashlib.sha256(content.encode()).hexdigest()

        # Extract filing metadata from path
        filing_type, filing_date = self._extract_metadata(file_path)

        return ParsedDocument(
            company_ticker=ticker,
            filing_type=filing_type,
            filing_date=filing_date,
            content=content,
            sections=sections,
            source_path=str(file_path),
            content_hash=content_hash,
            word_count=len(content.split())
        )

    def _parse_pdf(self, file_path: Path) -> str:
        """Extract text from PDF using pdfplumber."""
        text_parts = []

```

```python
        with pdfplumber.open(file_path) as pdf:
            for page in pdf.pages:
                text = page.extract_text()
                if text:
                    text_parts.append(text)

        return "\n\n".join(text_parts)

    def _parse_html(self, file_path: Path) -> str:
        """Extract text from HTML filing."""
        with open(file_path, "r", encoding="utf-8", errors="ignore") as f:
            soup = BeautifulSoup(f.read(), "html.parser")

        # Remove script and style elements
        for element in soup(["script", "style"]):
            element.decompose()

        # Get text
        text = soup.get_text(separator="\n")

        # Clean up whitespace
        lines = (line.strip() for line in text.splitlines())
        text = "\n".join(line for line in lines if line)

        return text

    def _extract_sections(self, content: str) -> dict[str, str]:
        """Extract key sections from 10-K content."""
        sections = {}
        content_upper = content.upper()

        for section_name, pattern in self.SECTION_PATTERNS.items():
            match = re.search(pattern, content_upper)
            if match:
                start = match.start()
                # Find end (next ITEM or end of document)
                next_item = re.search(r"ITEM\s*\d", content_upper[start + 100:])
                end = start + 100 + next_item.start() if next_item else len(content)
                sections[section_name] = content[start:end][:50000]  # Limit size

        return sections

    def _extract_metadata(self, file_path: Path) -> tuple[str, datetime]:
        """Extract filing type and date from file path."""
        # Path structure: .../ticker/filing_type/accession/file
        parts = file_path.parts
        filing_type = parts[-3] if len(parts) > 2 else "UNKNOWN"

        # Try to extract date from accession number (format: 0000000000-YY-NNNNNN)
        accession = parts[-2] if len(parts) > 1 else ""
```

```
        date_match = re.search(r"-(\d{2})-", accession)
        if date_match:
            year = int(date_match.group(1))
            year = 2000 + year if year < 50 else 1900 + year
            filing_date = datetime(year, 1, 1, tzinfo=timezone.utc)
        else:
            filing_date = datetime.now(timezone.utc)

        return filing_type, filing_date
```

3.4 Semantic Chunking

Documents must be chunked for later LLM processing and vector search:

```
from dataclasses import dataclass

@dataclass
class DocumentChunk:
    """A chunk of a document for processing."""
    document_id: str
    chunk_index: int
    content: str
    section: str | None
    start_char: int
    end_char: int
    word_count: int

class SemanticChunker:
    """Chunk documents with section awareness."""

    def __init__(
        self,
        chunk_size: int = 1000,       # Target words per chunk
        chunk_overlap: int = 100,     # Overlap in words
        min_chunk_size: int = 200     # Minimum chunk size
    ):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.min_chunk_size = min_chunk_size

    def chunk_document(
        self,
        doc: ParsedDocument
    ) -> list[DocumentChunk]:
        """Split document into overlapping chunks."""
        chunks = []

        # Chunk each section separately to preserve context
        for section_name, section_content in doc.sections.items():
            section_chunks = self._chunk_text(
                section_content,
                doc.content_hash,
                section_name
            )
```

```python
            chunks.extend(section_chunks)

        # Also chunk any remaining content
        if not doc.sections:
            chunks = self._chunk_text(doc.content, doc.content_hash, None)

        return chunks

    def _chunk_text(
        self,
        text: str,
        doc_id: str,
        section: str | None
    ) -> list[DocumentChunk]:
        """Split text into overlapping chunks."""
        words = text.split()
        chunks = []

        start_idx = 0
        chunk_index = 0

        while start_idx < len(words):
            end_idx = min(start_idx + self.chunk_size, len(words))

            # Don't create tiny final chunks
            if len(words) - end_idx < self.min_chunk_size:
                end_idx = len(words)

            chunk_words = words[start_idx:end_idx]
            chunk_content = " ".join(chunk_words)

            # Calculate character positions (approximate)
            start_char = len(" ".join(words[:start_idx]))
            end_char = start_char + len(chunk_content)

            chunks.append(DocumentChunk(
                document_id=doc_id,
                chunk_index=chunk_index,
                content=chunk_content,
                section=section,
                start_char=start_char,
                end_char=end_char,
                word_count=len(chunk_words)
            ))

            # Move forward with overlap
            start_idx = end_idx - self.chunk_overlap
            chunk_index += 1

            if end_idx >= len(words):
                break

        return chunks
```

3.5 Document Registry

Track all downloaded and processed documents:

```python
from pydantic import BaseModel, Field
from uuid import UUID, uuid4
from datetime import datetime, timezone
from enum import Enum

class DocumentStatus(str, Enum):
    PENDING = "pending"
    DOWNLOADED = "downloaded"
    PARSED = "parsed"
    CHUNKED = "chunked"
    INDEXED = "indexed"
    FAILED = "failed"

class DocumentRecord(BaseModel):
    """Metadata record for a document."""
    id: UUID = Field(default_factory=uuid4)
    company_id: UUID
    ticker: str
    filing_type: str
    filing_date: datetime
    source_url: str | None = None
    local_path: str | None = None
    s3_key: str | None = None
    content_hash: str | None = None
    word_count: int | None = None
    chunk_count: int | None = None
    status: DocumentStatus = DocumentStatus.PENDING
    error_message: str | None = None
    created_at: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc)
    )
    processed_at: datetime | None = None

    class Config:
        from_attributes = True
```

4 External Signals Pipeline (Lab 4)

Key Insight

The Four Signal Categories
PE firms use external signals to verify AI claims. Each signal category provides different evidence:

Signal	What It Reveals	Weight
Technology Hiring	Actual investment in AI talent	0.30
Innovation Activity	R&D commitment to AI	0.25
Digital Presence	Technology stack maturity	0.25
Leadership Signals	Executive commitment	0.20

4.1 Signal Data Models

```python
from pydantic import BaseModel, Field, model_validator
from uuid import UUID, uuid4
from datetime import datetime, timezone
from enum import Enum

class SignalCategory(str, Enum):
    TECHNOLOGY_HIRING = "technology_hiring"
    INNOVATION_ACTIVITY = "innovation_activity"
    DIGITAL_PRESENCE = "digital_presence"
    LEADERSHIP_SIGNALS = "leadership_signals"

class SignalSource(str, Enum):
    LINKEDIN = "linkedin"
    INDEED = "indeed"
    GLASSDOOR = "glassdoor"
    USPTO = "uspto"
    BUILTWITH = "builtwith"
    PRESS_RELEASE = "press_release"
    COMPANY_WEBSITE = "company_website"

class ExternalSignal(BaseModel):
    """A single external signal observation."""
    id: UUID = Field(default_factory=uuid4)
    company_id: UUID
    category: SignalCategory
    source: SignalSource
    signal_date: datetime
    raw_value: str                    # Original observation
    normalized_score: float = Field(ge=0, le=100)
    confidence: float = Field(default=0.8, ge=0, le=1)
    metadata: dict = Field(default_factory=dict)
    created_at: datetime = Field(
        default_factory=lambda: datetime.now(timezone.utc)
    )

```

```python
class CompanySignalSummary(BaseModel):
    """Aggregated signals for a company."""
    company_id: UUID
    ticker: str
    technology_hiring_score: float = Field(ge=0, le=100)
    innovation_activity_score: float = Field(ge=0, le=100)
    digital_presence_score: float = Field(ge=0, le=100)
    leadership_signals_score: float = Field(ge=0, le=100)
    composite_score: float = Field(ge=0, le=100)
    signal_count: int
    last_updated: datetime

    @model_validator(mode='after')
    def calculate_composite(self) -> 'CompanySignalSummary':
        """Calculate weighted composite score."""
        self.composite_score = (
            0.30 * self.technology_hiring_score +
            0.25 * self.innovation_activity_score +
            0.25 * self.digital_presence_score +
            0.20 * self.leadership_signals_score
        )
        return self
```

4.2 Job Posting Signal Collector

```python
import httpx
from bs4 import BeautifulSoup
import re
from dataclasses import dataclass

@dataclass
class JobPosting:
    """Represents a job posting."""
    title: str
    company: str
    location: str
    description: str
    posted_date: str | None
    source: str
    url: str
    is_ai_related: bool
    ai_skills: list[str]

class JobSignalCollector:
    """Collect job posting signals for AI hiring."""

    AI_KEYWORDS = [
        "machine learning", "ml engineer", "data scientist",
        "artificial intelligence", "deep learning", "nlp",
        "computer vision", "mlops", "ai engineer",
        "pytorch", "tensorflow", "llm", "large language model"
    ]

```

```python
    AI_SKILLS = [
        "python", "pytorch", "tensorflow", "scikit-learn",
        "spark", "hadoop", "kubernetes", "docker",
        "aws sagemaker", "azure ml", "gcp vertex",
        "huggingface", "langchain", "openai"
    ]

    def __init__(self):
        self.client = httpx.Client(
            timeout=30.0,
            headers={"User-Agent": "Mozilla/5.0 (compatible; research)"}
        )

    def analyze_job_postings(
        self,
        company: str,
        postings: list[JobPosting]
    ) -> ExternalSignal:
        """Analyze job postings to calculate hiring signal."""

        total_tech_jobs = len([p for p in postings if self._is_tech_job(p)
            ])
        ai_jobs = len([p for p in postings if p.is_ai_related])

        # Calculate metrics
        if total_tech_jobs > 0:
            ai_ratio = ai_jobs / total_tech_jobs
        else:
            ai_ratio = 0

        # Collect all AI skills mentioned
        all_skills = set()
        for posting in postings:
            all_skills.update(posting.ai_skills)

        # Score calculation (0-100)
        # - Base: AI ratio * 60 (max 60 points)
        # - Skill diversity: len(skills) / 10 * 20 (max 20 points)
        # - Volume bonus: min(ai_jobs / 5, 1) * 20 (max 20 points)
        score = (
            min(ai_ratio * 60, 60) +
            min(len(all_skills) / 10, 1) * 20 +
            min(ai_jobs / 5, 1) * 20
        )

        return ExternalSignal(
            company_id=None,  # Set by caller
            category=SignalCategory.TECHNOLOGY_HIRING,
            source=SignalSource.INDEED,
            signal_date=datetime.now(timezone.utc),
            raw_value=f"{ai_jobs}/{total_tech_jobs} AI jobs",
            normalized_score=round(score, 1),
            confidence=min(0.5 + total_tech_jobs / 100, 0.95),
            metadata={
```

```
                "total_tech_jobs": total_tech_jobs,
                "ai_jobs": ai_jobs,
                "ai_ratio": round(ai_ratio, 3),
                "skills_found": list(all_skills)
            }
        )

    def classify_posting(self, posting: JobPosting) -> JobPosting:
        """Classify a job posting as AI-related or not."""
        text = f"{posting.title} {posting.description}".lower()

        # Check for AI keywords
        is_ai = any(kw in text for kw in self.AI_KEYWORDS)

        # Extract AI skills
        skills = [skill for skill in self.AI_SKILLS if skill in text]

        posting.is_ai_related = is_ai
        posting.ai_skills = skills

        return posting

    def _is_tech_job(self, posting: JobPosting) -> bool:
        """Check if posting is a technology job."""
        tech_keywords = [
            "engineer", "developer", "programmer", "software",
            "data", "analyst", "scientist", "technical"
        ]
        title_lower = posting.title.lower()
        return any(kw in title_lower for kw in tech_keywords)
```

4.3 Technology Stack Signal Collector

```
@dataclass
class TechnologyDetection:
    """A detected technology."""
    name: str
    category: str
    is_ai_related: bool
    confidence: float

class TechStackCollector:
    """Analyze company technology stacks."""

    AI_TECHNOLOGIES = {
        # Cloud AI Services
        "aws sagemaker": "cloud_ml",
        "azure ml": "cloud_ml",
        "google vertex": "cloud_ml",
        "databricks": "cloud_ml",

        # ML Frameworks
        "tensorflow": "ml_framework",
```

```python
        "pytorch": "ml_framework",
        "scikit-learn": "ml_framework",

        # Data Infrastructure
        "snowflake": "data_platform",
        "databricks": "data_platform",
        "spark": "data_platform",

        # AI APIs
        "openai": "ai_api",
        "anthropic": "ai_api",
        "huggingface": "ai_api",
    }

    def analyze_tech_stack(
        self,
        company_id: UUID,
        technologies: list[TechnologyDetection]
    ) -> ExternalSignal:
        """Analyze technology stack for AI capabilities."""

        ai_techs = [t for t in technologies if t.is_ai_related]

        # Score by category
        categories_found = set(t.category for t in ai_techs)

        # Scoring:
        # - Each AI technology: 10 points (max 50)
        # - Each category covered: 12.5 points (max 50)
        tech_score = min(len(ai_techs) * 10, 50)
        category_score = min(len(categories_found) * 12.5, 50)

        score = tech_score + category_score

        return ExternalSignal(
            company_id=company_id,
            category=SignalCategory.DIGITAL_PRESENCE,
            source=SignalSource.BUILTWITH,
            signal_date=datetime.now(timezone.utc),
            raw_value=f"{len(ai_techs)} AI technologies detected",
            normalized_score=round(score, 1),
            confidence=0.85,
            metadata={
                "ai_technologies": [t.name for t in ai_techs],
                "categories": list(categories_found),
                "total_technologies": len(technologies)
            }
        )
```

4.4 Patent Signal Collector

```python
from dataclasses import dataclass
from datetime import datetime, timezone, timedelta
```

```python
from uuid import UUID

@dataclass
class Patent:
    """A patent record."""
    patent_number: str
    title: str
    abstract: str
    filing_date: datetime
    grant_date: datetime | None
    inventors: list[str]
    assignee: str
    is_ai_related: bool
    ai_categories: list[str]

class PatentSignalCollector:
    """Collect patent signals for AI innovation."""

    AI_PATENT_KEYWORDS = [
        "machine learning", "neural network", "deep learning",
        "artificial intelligence", "natural language processing",
        "computer vision", "reinforcement learning",
        "predictive model", "classification algorithm"
    ]

    AI_PATENT_CLASSES = [
        "706",  # Data processing: AI
        "382",  # Image analysis
        "704",  # Speech processing
    ]

    def analyze_patents(
        self,
        company_id: UUID,
        patents: list[Patent],
        years: int = 5
    ) -> ExternalSignal:
        """Analyze patent portfolio for AI innovation."""

        cutoff = datetime.now(timezone.utc) - timedelta(days=years * 365)
        recent_patents = [p for p in patents if p.filing_date > cutoff]
        ai_patents = [p for p in recent_patents if p.is_ai_related]

        # Scoring:
        # - AI patent count: 5 points each (max 50)
        # - Recency bonus: +2 per patent filed in last year (max 20)
        # - Category diversity: 10 points per category (max 30)

        last_year = datetime.now(timezone.utc) - timedelta(days=365)
        recent_ai = [p for p in ai_patents if p.filing_date > last_year]

        categories = set()
        for p in ai_patents:
            categories.update(p.ai_categories)
```

```python

        score = (
            min(len(ai_patents) * 5, 50) +
            min(len(recent_ai) * 2, 20) +
            min(len(categories) * 10, 30)
        )

        return ExternalSignal(
            company_id=company_id,
            category=SignalCategory.INNOVATION_ACTIVITY,
            source=SignalSource.USPTO,
            signal_date=datetime.now(timezone.utc),
            raw_value=f"{len(ai_patents)} AI patents in {years} years",
            normalized_score=round(score, 1),
            confidence=0.90,
            metadata={
                "total_patents": len(patents),
                "ai_patents": len(ai_patents),
                "recent_ai_patents": len(recent_ai),
                "ai_categories": list(categories)
            }
        )

    def classify_patent(self, patent: Patent) -> Patent:
        """Classify a patent as AI-related."""
        text = f"{patent.title} {patent.abstract}".lower()

        is_ai = any(kw in text for kw in self.AI_PATENT_KEYWORDS)

        categories = []
        if "neural network" in text or "deep learning" in text:
            categories.append("deep_learning")
        if "natural language" in text:
            categories.append("nlp")
        if "computer vision" in text or "image" in text:
            categories.append("computer_vision")
        if "predictive" in text:
            categories.append("predictive_analytics")

        patent.is_ai_related = is_ai or len(categories) > 0
        patent.ai_categories = categories

        return patent
```

5 Database Schema Extensions

Add these tables to your Snowflake schema:

```sql
-- Documents table
CREATE TABLE IF NOT EXISTS documents (
    id VARCHAR(36) PRIMARY KEY,
    company_id VARCHAR(36) NOT NULL REFERENCES companies(id),
    ticker VARCHAR(10) NOT NULL,
    filing_type VARCHAR(20) NOT NULL,
    filing_date DATE NOT NULL,
    source_url VARCHAR(500),
    local_path VARCHAR(500),
    s3_key VARCHAR(500),
    content_hash VARCHAR(64),
    word_count INT,
    chunk_count INT,
    status VARCHAR(20) DEFAULT 'pending'
        CHECK (status IN ('pending', 'downloaded', 'parsed',
                          'chunked', 'indexed', 'failed')),
    error_message VARCHAR(1000),
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
    processed_at TIMESTAMP_NTZ
);

-- Document chunks table
CREATE TABLE IF NOT EXISTS document_chunks (
    id VARCHAR(36) PRIMARY KEY,
    document_id VARCHAR(36) NOT NULL REFERENCES documents(id),
    chunk_index INT NOT NULL,
    content TEXT NOT NULL,
    section VARCHAR(50),
    start_char INT,
    end_char INT,
    word_count INT,
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP(),
    UNIQUE (document_id, chunk_index)
);

-- External signals table
CREATE TABLE IF NOT EXISTS external_signals (
    id VARCHAR(36) PRIMARY KEY,
    company_id VARCHAR(36) NOT NULL REFERENCES companies(id),
    category VARCHAR(30) NOT NULL
        CHECK (category IN ('technology_hiring', 'innovation_activity',
                            'digital_presence', 'leadership_signals')),
    source VARCHAR(30) NOT NULL,
    signal_date DATE NOT NULL,
    raw_value VARCHAR(500),
    normalized_score DECIMAL(5,2) CHECK (normalized_score BETWEEN 0 AND
        100),
    confidence DECIMAL(4,3) CHECK (confidence BETWEEN 0 AND 1),
    metadata VARIANT,  -- Snowflake JSON type
    created_at TIMESTAMP_NTZ DEFAULT CURRENT_TIMESTAMP()
```

```sql
);

-- Company signal summary (materialized view pattern)
CREATE TABLE IF NOT EXISTS company_signal_summaries (
    company_id VARCHAR(36) PRIMARY KEY REFERENCES companies(id),
    ticker VARCHAR(10) NOT NULL,
    technology_hiring_score DECIMAL(5,2),
    innovation_activity_score DECIMAL(5,2),
    digital_presence_score DECIMAL(5,2),
    leadership_signals_score DECIMAL(5,2),
    composite_score DECIMAL(5,2),
    signal_count INT,
    last_updated TIMESTAMP_NTZ
);

-- Indexes
CREATE INDEX IF NOT EXISTS idx_documents_company
    ON documents(company_id);
CREATE INDEX IF NOT EXISTS idx_documents_status
    ON documents(status);
CREATE INDEX IF NOT EXISTS idx_chunks_document
    ON document_chunks(document_id);
CREATE INDEX IF NOT EXISTS idx_signals_company
    ON external_signals(company_id);
CREATE INDEX IF NOT EXISTS idx_signals_category
    ON external_signals(category);
```

6 API Endpoints

Add these endpoints to expose evidence collection functionality:

Table 3: Evidence Collection API Endpoints

Method	Endpoint	Description
Documents		
POST	`/api/v1/documents/collect`	Trigger document collection for a company
GET	`/api/v1/documents`	List documents (filterable by company, type)
GET	`/api/v1/documents/{id}`	Get document with metadata
GET	`/api/v1/documents/{id}/chunks`	Get document chunks
Signals		
POST	`/api/v1/signals/collect`	Trigger signal collection for a company
GET	`/api/v1/signals`	List signals (filterable)
GET	`/api/v1/companies/{id}/signals`	Get signal summary for company
GET	`/api/v1/companies/{id}/signals/{category}`	Get signals by category
Evidence		
GET	`/api/v1/companies/{id}/evidence`	Get all evidence for a company
POST	`/api/v1/evidence/backfill`	Backfill evidence for all 10 companies
GET	`/api/v1/evidence/stats`	Get evidence collection statistics

6.1 Collection Endpoint Example

```python
from fastapi import APIRouter, BackgroundTasks, HTTPException
from pydantic import BaseModel
from uuid import UUID

router = APIRouter(prefix="/api/v1", tags=["evidence"])

class CollectionRequest(BaseModel):
    company_id: UUID
    filing_types: list[str] = ["10-K", "10-Q", "8-K"]
    years_back: int = 3

class CollectionResponse(BaseModel):
    task_id: str
    status: str
    message: str

@router.post("/documents/collect", response_model=CollectionResponse)
async def collect_documents(
    request: CollectionRequest,
    background_tasks: BackgroundTasks
):
```

```python
    """
    Trigger document collection for a company.

    This runs asynchronously - returns immediately with a task ID.
    """
    # Generate task ID
    task_id = str(uuid4())

    # Add to background tasks
    background_tasks.add_task(
        run_document_collection,
        task_id=task_id,
        company_id=request.company_id,
        filing_types=request.filing_types,
        years_back=request.years_back
    )

    return CollectionResponse(
        task_id=task_id,
        status="queued",
        message=f"Document collection started for company {request.company_id}"
    )

async def run_document_collection(
    task_id: str,
    company_id: UUID,
    filing_types: list[str],
    years_back: int
):
    """Background task for document collection."""
    # Implementation here
    pass
```

7 Evidence Collection Script

Create a main script to collect evidence for all 10 companies:

```python
#!/usr/bin/env python
"""
Collect evidence for all target companies.

Usage:
    python scripts/collect_evidence.py --companies all
    python scripts/collect_evidence.py --companies CAT,DE,UNH
"""

import argparse
import asyncio
from datetime import datetime, timezone
import structlog

from app.pipelines.sec_edgar import SECEdgarPipeline
from app.pipelines.document_parser import DocumentParser, SemanticChunker
from app.pipelines.job_signals import JobSignalCollector
from app.pipelines.tech_signals import TechStackCollector
from app.pipelines.patent_signals import PatentSignalCollector
from app.services.snowflake import SnowflakeService
from app.services.s3_storage import S3Storage

logger = structlog.get_logger()

TARGET_COMPANIES = {
    "CAT": {"name": "Caterpillar Inc.", "sector": "Manufacturing"},
    "DE": {"name": "Deere & Company", "sector": "Manufacturing"},
    "UNH": {"name": "UnitedHealth Group", "sector": "Healthcare"},
    "HCA": {"name": "HCA Healthcare", "sector": "Healthcare"},
    "ADP": {"name": "Automatic Data Processing", "sector": "Services"},
    "PAYX": {"name": "Paychex Inc.", "sector": "Services"},
    "WMT": {"name": "Walmart Inc.", "sector": "Retail"},
    "TGT": {"name": "Target Corporation", "sector": "Retail"},
    "JPM": {"name": "JPMorgan Chase", "sector": "Financial"},
    "GS": {"name": "Goldman Sachs", "sector": "Financial"},
}

async def collect_documents(ticker: str, db: SnowflakeService):
    """Collect SEC documents for a company."""
    logger.info("Collecting documents", ticker=ticker)

    pipeline = SECEdgarPipeline(
        company_name="PE-OrgAIR-Platform",
        email="your-email@university.edu"  # Replace with your email
    )

    parser = DocumentParser()
    chunker = SemanticChunker()

    # Download filings
```

```python
    filings = pipeline.download_filings(
        ticker=ticker,
        filing_types=["10-K", "10-Q", "8-K"],
        limit=10,
        after="2021-01-01"
    )

    logger.info("Downloaded filings", ticker=ticker, count=len(filings))

    # Parse and chunk each filing
    for filing_path in filings:
        try:
            doc = parser.parse_filing(filing_path, ticker)
            chunks = chunker.chunk_document(doc)

            # Store in database
            await db.insert_document(doc)
            await db.insert_chunks(chunks)

            logger.info(
                "Processed document",
                ticker=ticker,
                filing_type=doc.filing_type,
                chunks=len(chunks)
            )
        except Exception as e:
            logger.error("Failed to process", path=str(filing_path), error
               =str(e))

    return len(filings)

async def collect_signals(ticker: str, company_id: UUID, db:
   SnowflakeService):
    """Collect external signals for a company."""
    logger.info("Collecting signals", ticker=ticker)

    signals = []

    # Job postings (simplified - in practice, use API)
    job_collector = JobSignalCollector()
    # In real implementation, fetch actual job postings
    job_signal = job_collector.analyze_job_postings(
        company=TARGET_COMPANIES[ticker]["name"],
        postings=[]  # Would be populated from API
    )
    job_signal.company_id = company_id
    signals.append(job_signal)

    # Technology stack
    tech_collector = TechStackCollector()
    tech_signal = tech_collector.analyze_tech_stack(
        company_id=company_id,
        technologies=[]  # Would be populated from BuiltWith API
    )
```

```python
    signals.append(tech_signal)

    # Patents
    patent_collector = PatentSignalCollector()
    patent_signal = patent_collector.analyze_patents(
        company_id=company_id,
        patents=[]  # Would be populated from USPTO API
    )
    signals.append(patent_signal)

    # Store signals
    for signal in signals:
        await db.insert_signal(signal)

    # Update summary
    await db.update_signal_summary(company_id)

    logger.info("Collected signals", ticker=ticker, count=len(signals))

    return len(signals)

async def main(companies: list[str]):
    """Main collection routine."""
    db = SnowflakeService()

    stats = {
        "companies": 0,
        "documents": 0,
        "signals": 0,
        "errors": 0
    }

    for ticker in companies:
        if ticker not in TARGET_COMPANIES:
            logger.warning("Unknown ticker", ticker=ticker)
            continue

        try:
            # Ensure company exists in database
            company = await db.get_or_create_company(
                ticker=ticker,
                name=TARGET_COMPANIES[ticker]["name"],
                sector=TARGET_COMPANIES[ticker]["sector"]
            )

            # Collect documents
            doc_count = await collect_documents(ticker, db)
            stats["documents"] += doc_count

            # Collect signals
            signal_count = await collect_signals(ticker, company.id, db)
            stats["signals"] += signal_count

            stats["companies"] += 1
```

```python

        except Exception as e:
            logger.error("Failed to process company", ticker=ticker, error=str(e))
            stats["errors"] += 1

    logger.info("Collection complete", **stats)
    return stats

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--companies",
        default="all",
        help="Comma-separated tickers or 'all'"
    )
    args = parser.parse_args()

    if args.companies == "all":
        companies = list(TARGET_COMPANIES.keys())
    else:
        companies = [t.strip().upper() for t in args.companies.split(",")]

    asyncio.run(main(companies))
```

8 Evaluation Criteria

8.1 Requirements Checklist

Your submission must include all of the following components:

Requirements

All requirements must be completed for full credit (100 points)

SEC EDGAR Pipeline (35 points)

- ✓ Downloads 10-K, 10-Q, 8-K filings for all 10 companies
- ✓ Parses both PDF and HTML formats
- ✓ Extracts key sections (Item 1, 1A, 7)
- ✓ Implements semantic chunking with overlap
- ✓ Deduplicates using content hashing

External Signals Pipeline (30 points)

- ✓ Job posting signal collector implemented
- ✓ Technology stack signal collector implemented
- ✓ Patent signal collector implemented
- ✓ Signal scoring with normalization (0-100)
- ✓ Composite score calculation with weights

Data Persistence (20 points)

- ✓ Database schema extended (documents, chunks, signals)
- ✓ Document registry tracks all files
- ✓ Signals stored with metadata
- ✓ Company signal summaries computed

API & Integration (15 points)

- ✓ Collection endpoints functional
- ✓ Evidence retrieval endpoints working
- ✓ Background task processing
- ✓ Integrates with CS1 platform

8.2 Grading Rubric

Table 4: Detailed Grading Rubric

Component	Points
SEC EDGAR Pipeline (35 points)	
Filing download (10-K, 10-Q, 8-K)	10
Document parsing (PDF + HTML)	10
Section extraction	8
Semantic chunking	7
External Signals Pipeline (30 points)	
Job signal collector	10
Tech stack signal collector	8
Patent signal collector	7
Signal normalization & scoring	5
Data Persistence (20 points)	
Schema extensions (3 tables)	8
Document & chunk storage	6
Signal storage & summaries	6
API & Integration (15 points)	
Collection endpoints	6
Evidence retrieval endpoints	5
Background tasks	4
Total	**100**

9 Submission Requirements

9.1 What to Submit

1. **Updated GitHub Repository** containing:
   - All new pipeline code in `app/pipelines/`
   - New Pydantic models in `app/models/`
   - Database schema updates in `app/database/`
   - Collection script in `scripts/`
   - Updated `requirements.txt`
2. **Evidence Collection Report** (`docs/evidence_report.md`):
   - Summary statistics for each company
   - Document counts by filing type
   - Signal scores by category
   - Any data quality issues encountered
3. **Sample Evidence** (`data/samples/`):
   - 3 sample parsed documents (JSON format)
   - Signal summary for each company

9.2 Evidence Report Template

Your evidence report should include:

```
# Evidence Collection Report
## Case Study 2 | [Your Name]

### Summary Statistics
| Metric | Value |
|--------|-------|
| Companies processed | 10 |
| Total documents | XX |
| Total chunks | XX |
| Total signals | XX |

### Documents by Company
| Ticker | 10-K | 10-Q | 8-K | Total | Chunks |
|--------|------|------|-----|-------|--------|
| CAT    |      |      |     |       |        |
...

### Signal Scores by Company
| Ticker | Hiring | Innovation | Tech | Leadership | Composite |
|--------|--------|------------|------|------------|-----------|
| CAT    |        |            |      |            |           |
...

### Key Findings
1. [Your observations about say-do gaps]
2. [Patterns across sectors]
3. [Data quality issues encountered]
```

9.3 Submission Process

1. Push your final code to GitHub
2. Ensure the evidence report is complete
3. Submit the repository URL via the course LMS by **February 6, 3:59 PM**
4. Late submissions: 10% penalty per day, up to 3 days

10 Looking Ahead

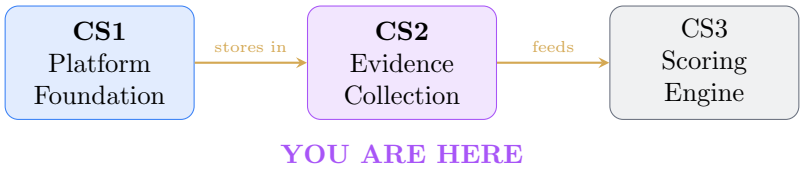

Figure 2: Your Progress Through the Course

What you're building now creates the evidence foundation:

- **Case Study 3** will use your SEC documents to extract AI evidence using GPT-4
- **Case Study 4** will add semantic search over your document chunks
- **Case Study 5** will automate evidence refresh with Airflow

Deliverable

CS2 Deliverable Summary
By completing this case study, you will have:

- 90+ SEC filings for 10 companies
- Documents chunked and ready for LLM processing
- External signals quantifying AI hiring, patents, and tech stack
- Unified evidence schema connecting internal and external data

"SEC filings show intent.
External signals show execution.
You need both."

Good luck! — Sri Krishnamurthy